Foundation Models for PDEs in fluid dynamics



M2 Master's Thesis

David Nelischer

Supervisor : Prof. Siddhartha Mishra

Seminar for Applied Mathematics

D-MATH

01/04/2025 - 17/07/2025

M2 Physics of Complex Systems - Paris-Saclay University





Contents

1 Introduction **1**

2 POSEIDON: Foundation Model for PDEs **2**

2.1 Operator Learning Task 2

2.2 Model Architecture 2

2.3 Training and Inference 3

3 Contrastive Learning Framework **4**

3.1 Representation Learning 4

3.2 Theoretical Definition 5

3.3 Training 6

4 Approach **6**

4.1 Methodology 6

4.2 Implementation Approach 7

5 Experiments **9**

5.1 Datasets 9

5.2 In-distribution and Out-of-distribution Data. 10

5.3 Main Results 11

5.4 Analysis and Discussion 14

6 Conclusion **16**

A Appendix **19**

A.1 Parameters 19

A.2 Predictions 19

1 Introduction

This work represents the culmination of the first three months of my master's thesis at the Seminar for Applied Mathematics at ETH Zürich. Having pursued previous research in numerical physics during my bachelor's thesis at the LPENS and my previous master's work at the University of Oxford, I was particularly drawn to exploring the intersection of numerical methods and physics. My prior experience with turbulent flow simulations at ENS, where I applied finite element methods to solve complex 3D grid dynamics, provided valuable context for appreciating the potential of ML-based approaches to PDE solving.

Context. Partial Differential Equations (PDEs) are essential mathematical models across the sciences. Solving a PDE consists in computing (often approximately) the *solution operator* that maps function space inputs (initial and boundary conditions, coefficients, sources) to the PDE solution in a certain function space. A number of well-established numerical methods, such as finite differences, finite element or finite volume schemes have been very successfully used to approximate PDE solution operators in the last decades [1]. These methods however have significant computational costs, particularly in high dimensions. More recently, *data-driven machine learning* methods for solving PDEs have emerged as fast and accurate alternatives [2] and offer significant advantages in terms of computational efficiency, robustness and accuracy. Within this emerging field, methods aiming to learn the underlying PDE solution operator, known as *operator learning* represent a particularly promising approach [3].

However, the requirement of a large number of training samples to effectively learn target solution operators presents a substantial barrier to the practical deployment for existing operator learning models. *Sample efficiency* is especially relevant in scenarios where generating training data is computationally expensive or where rapid adaptation to new physics is required. Similar limitations arise in language modeling or computer vision and have been successfully addressed using *foundation models* [4]. These generalist models are first trained on a large set of data distributions during the so called *pretraining* phase. Leveraging the ability of neural networks to learn *effective representations* during pretraining, the models are then deployed on downstream tasks by performing *finetuning* on a few task specific samples. The ability of foundation models to *generalize* to unseen and unrelated PDEs and data-distributions via finetuning remains a fundamental and unanswered science question.

Building on the paradigms of foundation models, Siddhartha Mishra and his group have proposed POSEIDON [5], a foundation model for learning the solution operators of PDEs. Based on a novel training strategy for time-dependent PDEs, POSEIDON achieves multiple orders of magnitude improvement in sample efficiency over existing PDE operator learning models while enabling transfer to unseen physics.

Research Objectives. Building upon the POSEIDON model, this thesis aims to enhance the models

performance through the implementation of a novel representation learning framework inspired by the Rank-N-Contrast method introduced by Zha et al. [6]. The primary objective is to enable POSEIDON to learn continuous representations of input data. As this continuity reflects the underlying physical properties, the model should gain a deeper understanding of both the data structure and its governing dynamics. Continuous representations provide smooth decision boundaries that are less susceptible to perturbations in data samples and enable better interpolation between training samples. By capturing the underlying mathematical relationships governing the data, the model becomes less dependent on specific training examples and more capable of handling variations and noise in the data. This should in turn enhance robustness and generalization capabilities [7]. I will first introduce the model as well as the contrastive learning framework before detailing my approach and discussing results.

2 POSEIDON: Foundation Model for PDEs

2.1 Operator Learning Task

The PDE solution operators are treated as *function-to-function mappings* in continuous function spaces $\mathcal{X} \subset C(D;\mathbb{R}^n)$ where D (here $D = [0,1]^2$) is the spatial domain. These operators map function space inputs (initial and boundary conditions, coefficients, sources) to the PDE solution. The underlying *operator learning task* that POSEIDON addresses can be formulated as follows: We denote a generic time-dependent PDE as

$$\begin{aligned} &\partial_t u(x,t) + \mathcal{L}\left(u, \nabla_x u, \nabla_x^2 u, \ldots\right) = 0, \quad \forall x \in D \subset \mathbb{R}^d, t \in (0,T), \\ &\mathcal{B}(u) = 0, \quad \forall (x,t) \in \partial D \times (0,T), \quad u(0,x) = a(x), \quad x \in D \end{aligned} \tag{1}$$

with $u \in C([0,T];\mathcal{X})$ the solution of (1) and $\mathcal{L}, \mathcal{B}$ the underlying differential and boundary operators. The goal is to learn an approximation $\mathcal{S}^* \approx \mathcal{S}$ of the solution operator $\mathcal{S} : [0,T] \times \mathcal{X} \mapsto \mathcal{X}$ such that given an initial datum $a \in \mathcal{X}$, the function $u(t) = \mathcal{S}(t,a)$ is the solution of (1) at any time $t \in [0,T]$. The learned operator must generate the entire solution trajectory $\{\mathcal{S}^*(t,a)\}$ for all $t \in [0,T]$.

2.2 Model Architecture

The backbone for the model is the scalable Operator Transformer (scOT), introduced by Liu et al. [8], see Figure 1 (a-c) for an illustrated summary.

Patch Embedding. Function inputs $a \in \mathcal{X}$ and lead time t are processed through a *multiscale vision transformer* with *lead-time conditioning*. The input is first partitioned into patches and linearly embedded into a latent space through the *patch partitioning and embedding operator* $\mathbf{v} = \hat{E}(a)$, with $\hat{E}$ defined in [8]. The operator $\hat{E}$ transforms the input function into a piecewise constant function within spatial patches by using weighted averages, then maps these values into a C-dimensional latent space resulting in $\mathbf{v} \in C(D;\mathbb{R}^C)$.

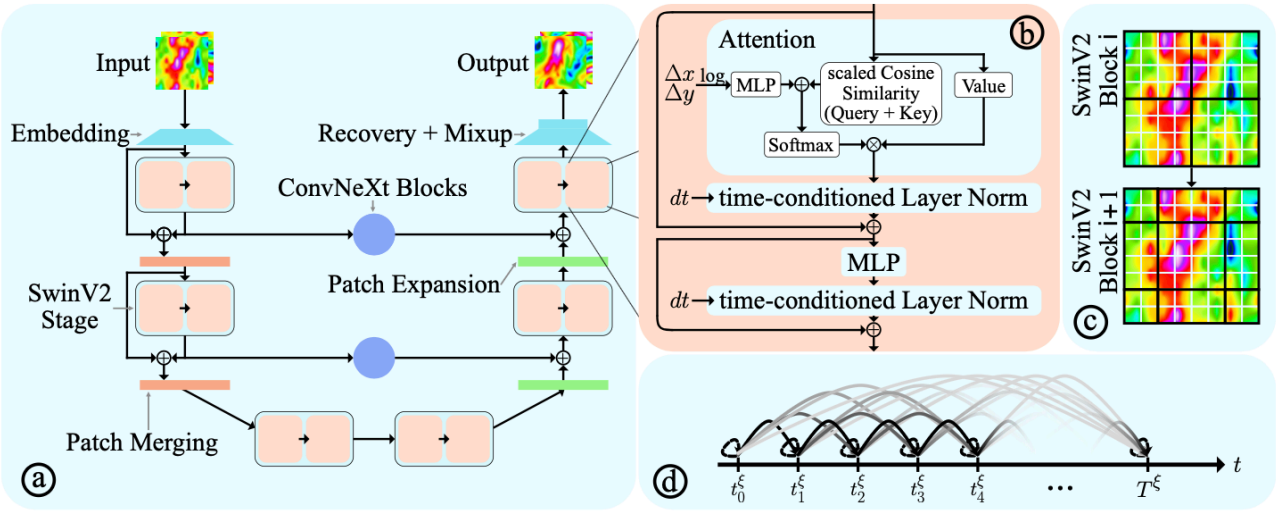


Figure 1: (a) scOT model underlying POSEIDON; (b) SwinV2 Transformer block; (c) Shifting Window over patch-based tokens with window (patch) boundaries in black (white); (d) all2all Training for time-dependent PDEs. (Figure taken from [5])

SwinV2 Transformer. The embedded patches are processed through a sequence of *SwinV2 transformer* blocks introduced in Liu et al. [8], see 1 (b-c). In particular, the attention operator (defined in **SM** 2) acts only inside of each window, which is a coarser subdivision of D, improving computational efficiency as compared to standard self-attention mechanisms. The windows are *shifted across layers* to ensure that all points in the domain can attend to each other through hierarchical processing. The *lead-time conditioning* mechanism allows for *continuous-in-time evaluations* as opposed to discrete training timestamps, see [5]. These transformer blocks are arranged in a hierarchical U-Net style *encoder-decoder* architecture [9], with the encoder employing patch merging operations (downsampling) to capture coarse-grained features, and the decoder patch expansion operations (upsampling) to recover fine-grained details. Layers at the same scale within the encoder and decoder are connected through *ConvNeXt* convolutional layers [10], enabling the preservation of both local differential operators and global solution behavior.

2.3 Training and Inference

Loss Function and all2all Training The scOT model can be denoted as $\mathcal{S}_\theta^* : [0,T] \times \mathcal{X} \mapsto \mathcal{X}$ with trainable model parameters $\theta \in \Theta \subset \mathbb{R}^p$. The aim of the training is to approximate the solution operator $\mathcal{S}$ of (1), by finding the set of model parameters that minimize the mismatch between model predictions and ground truth training data. The training data is given as trajectories of solutions of (1) $\{\mathcal{S}(t_k, a_i)\}$ for $0 \le k \le K$, $1 \le i \le M$ and $0 = t_0 < t_1 < \ldots t_k < \ldots < t_K = T$. Here t_k represents discrete time points and a_i the initial condition and M the total number of training trajectories. In order to maximize training data utilization, a novel *all2all training* strategy is employed. This training strategy uses the fact that any approximate solution $u(t) = \mathcal{S}(t,a)$ of (1) at time t' can be obtained as $\mathcal{S}(t'-t, u(t))$ for any intermediate time $t < t'$.

The loss function is defined as:

$$\widehat{\mathcal{L}}(\theta) := \frac{1}{M\widehat{K}} \sum_{i=1}^{M} \sum_{k,\bar{k}=0,k\leq\bar{k}}^{K} \left\| \mathcal{S}\left(t_{\bar{k}} - t_k, u_i\left(t_k\right)\right) - \mathcal{S}_\theta^*\left(t_{\bar{k}} - t_k, u_i\left(t_k\right)\right) \right\|_{L^p(D)}^p, \tag{2}$$

where $u_i(t_k) = S(t_k, a_i)$ represents the solution at time t_k for initial condition a_i, and $\bar{K} = \frac{(K+1)(K+2)}{2}$. For a given trajectory of solutions, the *all2all training* strategy thus considers all possible pairs of data points $(u(t_k), u(t_{\bar{k}}))$ within the trajectory where $k \neq \bar{k}$, see Figure 1 (d) for an illustration. This effectively increases the number of training pairs from $O(K)$ to $O(K^2)$ per trajectory while maintaining the physical consistency of the underlying PDE dynamics.

Pretraining and Finetuning. The original pretraining dataset in [5] consists of trajectories generated from six solution operators, including four from the compressible Euler equations and two from the incompressible Navier-Stokes equations, chosen to capture a wide range of physical phenomena, such as shocks, turbulence, and mixing layers. During pretraining, the model parameters, θ, are optimized to minimize the mismatch between the model's predictions and the ground truth across all operators and data distributions in the dataset.

During the finetuning phase, the model is trained on specific downstream tasks to adapt to new PDEs and data distributions. A subset of the model parameters is initialized with the pretrained weights, while others are trained from scratch. The finetuning process involves a small number of *task-specific* training examples. This approach allows the model to leverage the representations learned during pretraining while adapting to the unique characteristics of the downstream task.

3 Contrastive Learning Framework

3.1 Representation Learning

Motivation. The learning of meaningful feature embeddings is crucial as it allows the model to capture underlying data structures, patterns and relationships. This process know as *representation learning* enables better performance across diverse downstream tasks, improves model generalization to unseen data, facilitates transfer learning between domains and makes models more robust to variations and noise in the input [11]. As presented in Section 2.3, POSEIDONs current training approach optimizes an end-to-end model using the L_1 distance or *absolute error* between the model predictions and ground-truth targets. This approach, however, does not explicitly emphasize the learning of *regression-aware representations* that capture the continuous nature of the target space. The key observation motivating this thesis is that the L_1 loss used currently in the POSEIDON setup has been shown to *learn fragmented representations* that do not preserve inherent continuities among the training data.

Research Objectives. Building on the contrastive learning framework introduced by Zha et al.

[6], I aim to implement a regression-aware training approach within the POSEIDON framework. This would allow the model to *learn and preserve an intrinsic ordering* in the data by creating a continuous latent space. In a sense, the model would gain *a more physical understanding* of the training data by embedding samples such that distances in the latent space reflect meaningful relationships within the data. This way, we hope to achieve an overall better performance as well as better generalization to and robustness on unseen data.

3.2 Theoretical Definition

In their paper, Zha et al. [6] introduce *Rank-N-Contrast*, a novel design of a contrastive loss function $\mathcal{L}_{\text{RNC}}$, which enforces a continuous structure in the feature space that aligns with the continuous nature of the regression targets.

Ordering of the Feature Space: The RNC framework enforces a specific order constraint in the learned feature space that directly corresponds to the ordering in the label space. Given a batch of N input-label pairs $\{(x_n, y_n)\}_{n\in[N]}$, the aim is to learn the embeddings $v_\ell = f(x_\ell) \in \mathbb{R}^{d_e}$ using the RNC loss. For an anchor embedding v_i and any other embedding v_j, one defines the set $S_{i,j} := \{v_k | k \neq i, d(y_i, y_k) \geq d(y_i, y_j)\}$ as the set of samples with higher ranks than v_j in terms of the distance $d(.,.)$ to v_i. Here $d(.,.)$ is the distance measure between two labels (e.g. L_1 distance). The RNC approach ranks samples in a batch according to their label distances from an anchor, then contrasts each sample against all others with higher ranks. For samples with distinct label distances to an anchor, the feature space similarities must maintain the same relative ordering. This ensures that closer samples in the label space become more similar in the learned feature space, see Figure 2 for an illustration.

This order constraint can be expressed mathematically as follows, where $\text{sim}(\cdot,\cdot)$ represents the similarity measure between embeddings, see Section 4.2: $d(y_i, y_j) < d(y_i, y_k) \Rightarrow \text{sim}(v_i, v_j) > \text{sim}(v_i, v_k)$. The theoretical analysis provided by Zha et al. [6] proves that optimizing the RNC loss results in δ*-ordered feature embeddings*, where the order constraint is satisfied up to a tolerance parameter δ.

Mathematical Formulation. The normalized likelihood of v_j given v_i and $S_{i,j}$ is given by:

$$P(v_j | v_i, S_{i,j}) = \frac{\exp(\text{sim}(v_i, v_j)/\tau)}{\sum_{v_k \in S_{i,j}} \exp(\text{sim}(v_i, v_k)/\tau)} \tag{3}$$

where τ is a temperature parameter. By maximizing $P(v_j | v_i, S_{i,j})$, one effectively increases the probability that v_j outperforms all other samples in the set $S_{i,j}$ and emerges at the top rank within this set. By iterating this process over all possible anchor-sample pairs in the batch, the method guarantees that the learned embeddings preserve the relative ordering present in the continuous label space. The RNC approach includes a temperature parameter τ that controls the sharpness of the probability distribution in the loss function and therefore the contrast sensitivity.

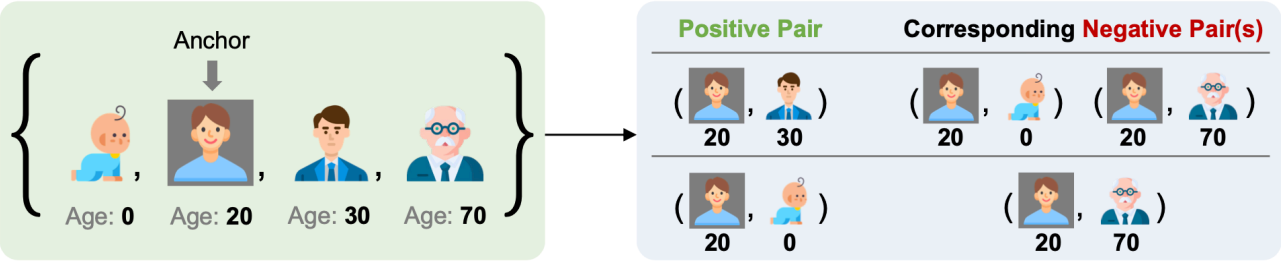


Figure 2: **Illustration of $\mathcal{L}_{\text{RNC}}$ with positive and negative pairs.** (left) Sample batch containing input data with corresponding age labels. (right) Positive-negative pair construction with the 20-year-old man as anchor (highlighted in gray). For the positive pair (anchor, 30-year-old), the label distance is 10, making samples with distances ≥ 10 (0-year-old baby and 70-year-old man) serve as negatives. For the positive pair (anchor, 0-year-old), only the 70-year-old man has a larger label distance ($50 > 20$), thus becoming the only negative sample. Figure taken from [6]

3.3 Training

The per-sample RnC loss is defined as the *average negative log-likelihood* over all other samples:

$$l_i^{(\text{RNC})} = \frac{1}{2N-1} \sum_{j=1, j\neq i}^{2N} -\log \frac{\exp(\text{sim}(v_i, v_j)/\tau)}{\sum_{v_k \in S_{i,j}} \exp(\text{sim}(v_i, v_k)/\tau)} \tag{4}$$

The RnC loss is obtained by aggregating over all anchors:

$$\mathcal{L}_{\text{RNC}} = \frac{1}{2N} \sum_{i=1}^{2N} l_i^{(\text{RNC})} \tag{5}$$

Given a regression task and a simple neural network model composed of a feature encoder $f : X \to \mathbb{R}^{d_e}$ and a predictor $g : \mathbb{R}^{d_e} \to \mathbb{R}^{d_t}$, Zha et al. [6] aim to predict the target $y \in \mathbb{R}^{d_t}$ based on the input data $x \in X$ using the $\mathcal{L}_{\text{RNC}}$ loss in the following two-step approach:

1. **Representation Learning:** Train a feature encoder $f : \mathcal{X} \to \mathbb{R}^{d_e}$ using $\mathcal{L}_{\text{RNC}}$.
2. **Regression:** Freeze the encoder and train a predictor $g : \mathbb{R}^{d_e} \to \mathbb{R}^{d_t}$ using a standard regression loss.

4 Approach

4.1 Methodology

The main motivation of our approach were t-SNE plots of the POSEIDON latent space showing a fragmented latent space structure as seen on Figure 6 when using the L_1 regression loss. This observation suggested the use of a regression aware representation learning approach such as the RnC approach. Ideally, this could lead to an increase in performance and generalization as shown in the paper by presented by Zha et al. [6]. However, POSEIDON necessitated a highly tailored

approach due to the complexity of its transformer architecture. Below is a brief overview of the approach I took, roughly in order:

- **Dataset Creation:**
  Generation of a training dataset, compatible with the all2all training scheme, to serve as a baseline example / proof of concept for the contrastive learning approach with POSEIDON. The dataset is based on the Wave equation (6) and allows for the parameterization of the number of Fourier modes to serve as the continuous parameter to vary in the data. Currently I am working on incompressible Navier-Stokes (NS) equations (9) that allow for a more complex behavior including turbulent features and mixing layers.

- **L_1 and L_2 baseline on the new datasets:**
  Training and fine-tuning of POSEIDON on the new datasets to obtain baseline values for both the L_1 and the L_2 loss. Grid search over hyperparameters until achieving a satisfactory performance (L_1 average error around 2%) on the dataset and validation of the generated predictions. Performance metrics on the in and out-of-distribution validation data for the standard L_1 and L_2 loss.

- **Contrastive learning approach within the POSEIDON framework:**
  Due to the higher complexity of POSEIDON and the underlying scOT architecture as compared to the transformer used by Zha et al. [6], I had to implement a diverging approach while maintaining the mathematical formulation of the loss function, see Section 4.2.

- **Training and hyperparameter tuning of POSEIDON using $\mathcal{L}_{\mathbf{RNC}}$:**
  Training and fine-tuning of the POSEIDON model on the new datasets using my implementation of $\mathcal{L}_{\text{RNC}}$. Implementation of a distributed training approach to allow for larger batch sizes.

- **Performance metrics on the in and out-of distribution data using $\mathcal{L}_{\mathbf{RNC}}$:**
  Experiments on the Wave equation and incompressible NS datasets (5) to evaluate both in-distribution and out-of-distribution performance of the novel contrastive learning approach as compared to the regression loss.

4.2 Implementation Approach

The deep learning architecture used by Zha et al. [6] has been implemented using ResNet-18 [12] as a backbone architecture, and a one dimensional target-space in mind. This proved to be a challenge as their feature space dimension was smaller (512 for ResNet-18 **vs** 12k for the smallest T variant of scOT). I therefore based my approach on the mathematical formulation of the RnC loss, but diverged from their implementation in the following ways:

Dimensionality Reduction in the feature space

To compute the similarity measure in the feature space of scOT, I implemented the following dimensionality reduction beforehand on the output of the last decoder layer:
$f : \mathbb{R}^d \to \mathbb{R}^{d_{\text{out}}}$, 2-stage approach with intermediate dimension $d_{\text{int}} = 1024$. The first stage maps to d_{int} including a GELU activation function. The second stage projects to $\mathbb{R}^{d_{\text{out}}}$ followed by an L_2 layer normalization. All weights are initialized as orthogonal. This transformed feature space dimension becomes a parameter in the code, for which dim = 512 provided the best performance.
Additionally, this provided an improvement to the model as I was now able to process batch sizes of up to 512. Given that contrastive loss inherently requires larger batch sizes, this increased the performance of the model by about 3% of L_1 error on average between runs.

Feature Similarity Measure

The two standard approaches I implemented are the cosine similarity measure and the negative L2 norm (i.e., $-|v_i - v_j|_2$) [11]. The latter proved to be more expressive as it takes into account both the direction and the magnitude of the feature vectors as opposed to only the direction for the cosine similarity measure.

Temperature Parameter τ

The finetuning of this parameter proved to be critical for the convergence of the model. Besides a fixed τ value, I experimented with varying the parameter over a specific interval across training steps. I ended up finetuning the value to $\tau = 1.75$ and choosing the fixed τ version.

Label Difference

As the initial paper computes label distances in a one-dimensional target space, I experimented using L_1 and L_2 norms in the 2D target space (velocity fields) and found the L_2 norm to work best.

Reconstruction Loss

One major change in my approach compared to the RNC paper has been the implementation of an additional *reconstruction loss* for the target space, combined with the contrastive loss. This has been shown to improve my training performance by about 3% of L_1 error on average. For the implementation of the reconstruction (recon) loss, I opted for an L_1 loss and found the optimal balance for the weighting of both losses to be at 90% contrastive + 10% recon. This balance is critical as a larger recon loss would hinder the effects of the contrastive loss. The recon loss indeed only provides stability (especially in the first iterations of the training) for the contrastive loss and is not meant to impede on its performance.

Training Approach

The implementation of the reconstruction loss allows for a *one-stage training* approach as opposed to the two-stage approach opted for by [6]. I implemented a *distributed training* approach to allow for larger batch sizes that benefit the contrastive learning, see Section 5.4.

5 Experiments

5.1 Datasets

Wave equation. The dataset is based on the following Wave equation

$$u_{tt} - c^2 \Delta u = 0, \text{ in } D \times (0, T), \quad u_0(x, y) = f(x, y), \tag{6}$$

with a constant propagation speed $c = 0.1$ and initial condition f given by:

$$f(x, y) = \frac{\pi}{K^2} \sum_{i,j=1}^{K} a_{ij} \cdot \left(i^2 + j^2\right)^{-r} \sin(\pi i x) \sin(\pi j y), \quad \forall (x, y) \in D \tag{7}$$

We take $D = [0, 1]^2$, $r = 1$ and the coefficients a_{ij} are randomly sampled from a normal distribution for each trajectory. The integer parameter K defines the number of Fourier modes in the initial condition f and varies in $[18, 28]$ as specified in Section 5.2. Each trajectory is computed using the analytical solution u given by

$$u(x, y, t) = \frac{\pi}{K^2} \sum_{i,j}^{K} a_{ij} \cdot \left(i^2 + j^2\right)^{-r} \sin(\pi i x) \sin(\pi j y) \cos\left(c\pi t \sqrt{i^2 + j^2}\right), \quad \forall (x, y) \in D \tag{8}$$

The trajectories are computed on a 64×64 grid over 10 time steps ranging from $t = 0$ to $t = 5$. Both the time steps and the velocity fields are normalized during training. The dataset is generated to be compatible with the *all2all* training approach developed in [5].

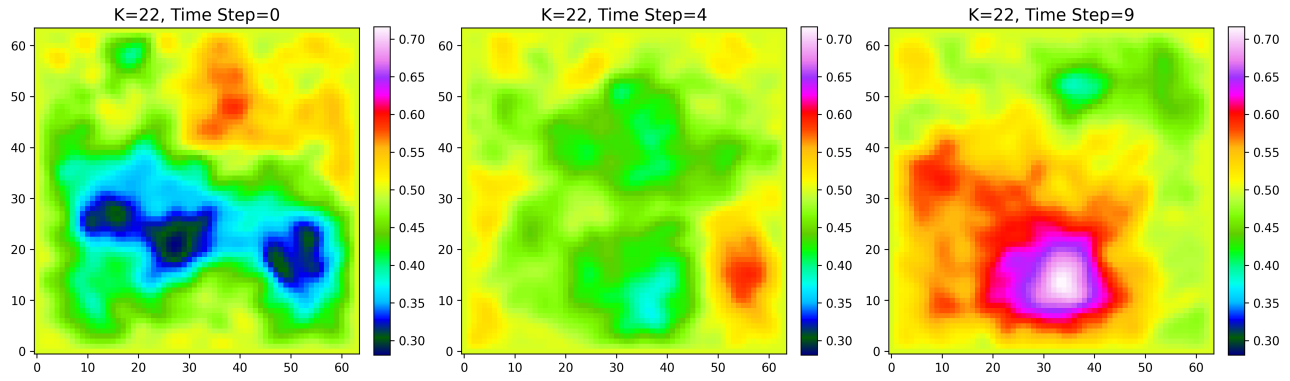


Figure 3: **Wave equation.** Analytical solution u for $K = 22$ at different time steps between $t = 0$ and $t = 5$.

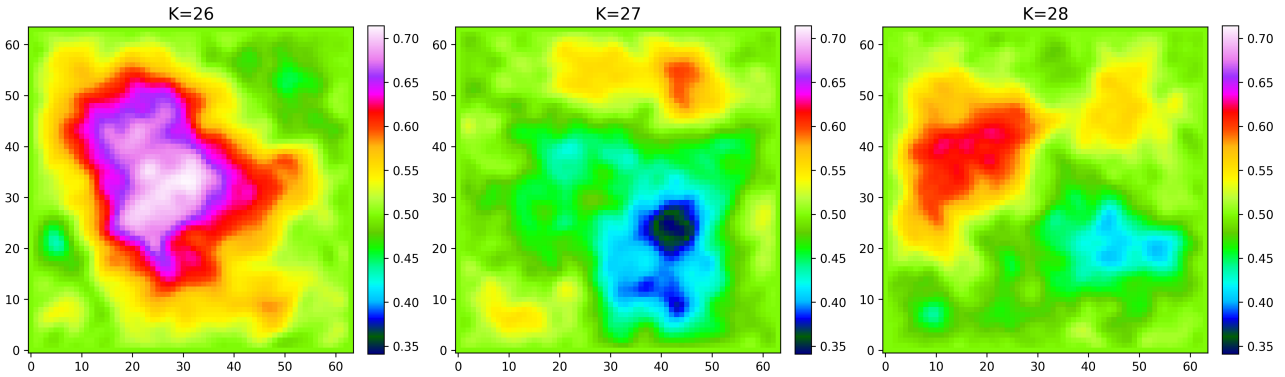


Figure 4: **Wave equation.** Initial condition f for different values of K at initial time step $t = 0$.

Navier-Stokes equations. This datasets considers the *incompressible Navier-Stokes equation*

$$u_t + (u \cdot \nabla) u + \nabla p = \nu \Delta u, \quad \operatorname{div} u = 0, \tag{9}$$

in the domain $D = [0,1]^2$ with $u : [0,T] \times D \mapsto \mathbb{R}^2$ the velocity field, $p : [0,T] \times D \mapsto \mathbb{R}_+$ the pressure and $\nu = 4 \times 10^{-4}$ a small viscosity. The training trajectories are approximated over 10 time steps ranging from $t = 0$ to $t = 5$, using a *spectral method* on a 128×128 grid. I consider the following initial conditions to allow for a range of dynamics, the exact equations are given in [5]:

- **NS-Sines:** Velocity fields u_x and u_y given as linear combinations of sines and cosines.
- **NS-Gauss:** Vorticity field given as a superposition of a large number of Gaussians.
- **NS-PwC:** Initial condition on the vorticity assumed to be constant along a uniform partition of the underlying domain. Trajectory given in Figure 5.

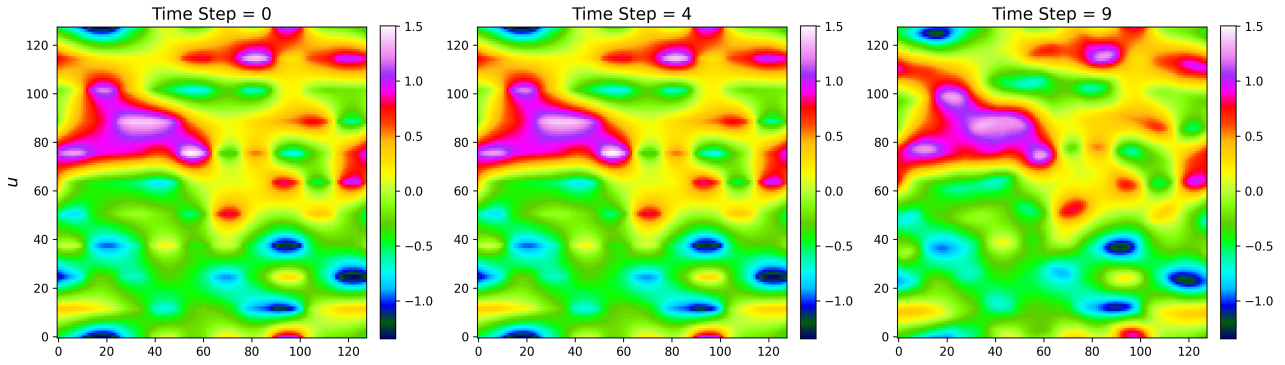


Figure 5: **NS-PwC.** Numerical approximation u of the incompressible Navier-Stokes equation at different time steps between $t = 0$ and $t = 5$. The initial condition is of type NS-PwC.

5.2 In-distribution and Out-of-distribution Data.

Out-of-distribution (OOD) prediction tasks offer an interesting setup as any "understanding" of physics will be particularly reflected in the models performance. They consist in training the model

on a given data distribution and evaluating its performance on unseen data. The out-of-distribution data can be chosen arbitrarily close to the in-distribution data I conducted several experiments to evaluate the performance on out-of-distribution prediction tasks using both the contrastive and standard regression approach.

Wave equation. As a standard evaluation setup, the number of Fourier modes K is chosen to be in $[18, 25]$ for the in-distribution (ID) data and in $[26, 28]$ for the out-of-distribution data. Figure 3 gives three snapshots from an ID trajectory and Figure 4 initial conditions for different K values in the OOD range.
The training data consist of 1024 trajectories and the validation data of 512 trajectories. Both training and validation use the all2all approach on all ten time steps, leading to *over 45 thousand* training pairs.

Navier-Stokes equations. The pre-training of the model is done on the *combined* NS-Sines and NS-Gauss datasets. The OOD evaluation is performed on the NS-PwC dataset, evaluating the *zero-shot* performance, unseen data in training with different flow dynamics, of the model. All three datasets consist of 1024 trajectories having seven time steps and use the all2all approach for training and evaluation.

5.3 Main Results

I measured the performance of the contrastive learning approach against the baseline regression approach (L_1 regression loss) using the smallest T variant of POSEIDON that has $\approx$ 21M parameters. The exact model specifications are given in [5] as well as the parameter choices for training and inference. The results are averaged on the entire validation dataset.

	In-distribution			**Out-of-distribution**		
Method	**Average**	**Min**	**Max**	**Average**	**Min**	**Max**
POSEIDON-T L_1	2.03	0.77	6.26	1.44	0.80	4.08
POSEIDON-T $\mathcal{L}_{\text{RNC}}$	1.34	0.50	5.99	0.912	0.49	3.91
Gains	**+0.69**	**+0.27**	**+0.27**	**+0.53**	**+0.31**	**+0.17**

Table 1: **Wave equation dataset.** ID and OOD comparison between the RNC loss and the L_1 regression loss. The contrastive learning learning approach surpasses the regression loss in both scenarios for this dataset.

Latent space. Figure 6 shows the t-SNE plots of the POSEIDON model latent space after training using the $L1$ regression loss and the contrastive learning approach. The Figure clearly highlights the continuous ordering of the latent space with regards to the parameter K, achieved using the contrastive learning approach.
Predictions. The results of Table 1 are directly reflected in the accuracy of the predictions generated.

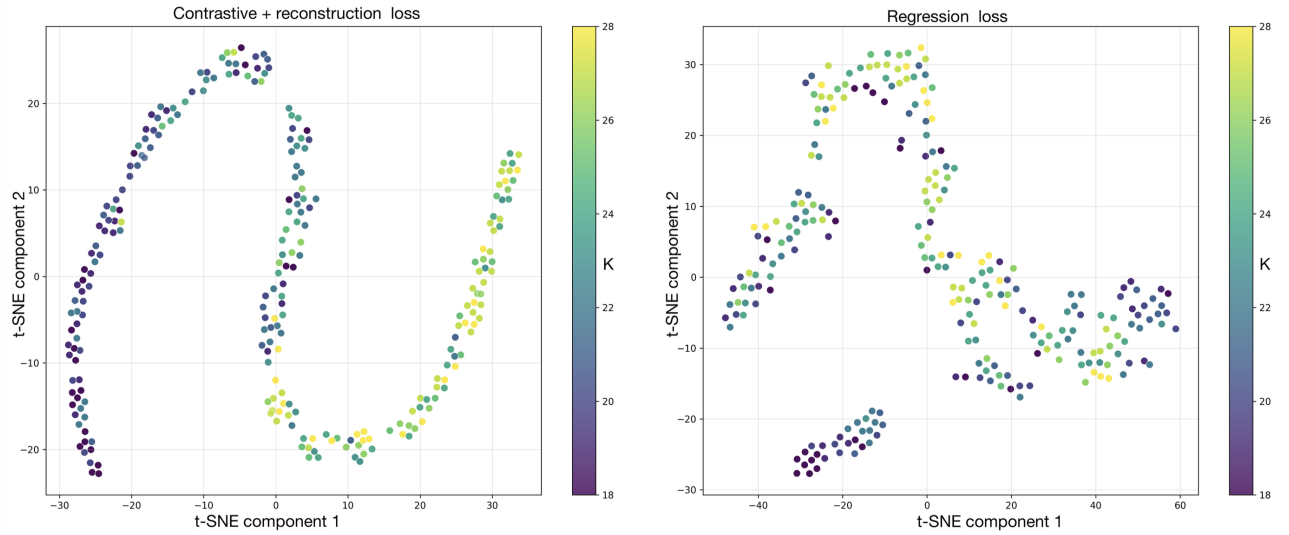


Figure 6: **Learned representations on the Wave equation datasets.** The regression learning (L_1) (right) fails to recognize the underlying continuous information in the data. In contrast, the contrastive approach (left) captures the intrinsic continuity in the K parameter.

Figures 7-10 show predictions of the POSEIDON-T model using the contrastive approach and the L_1 regression loss. Additional predictions are given in **SM** (A.2). The difference of both approaches is visible in graphical details such as boundaries of different regions.

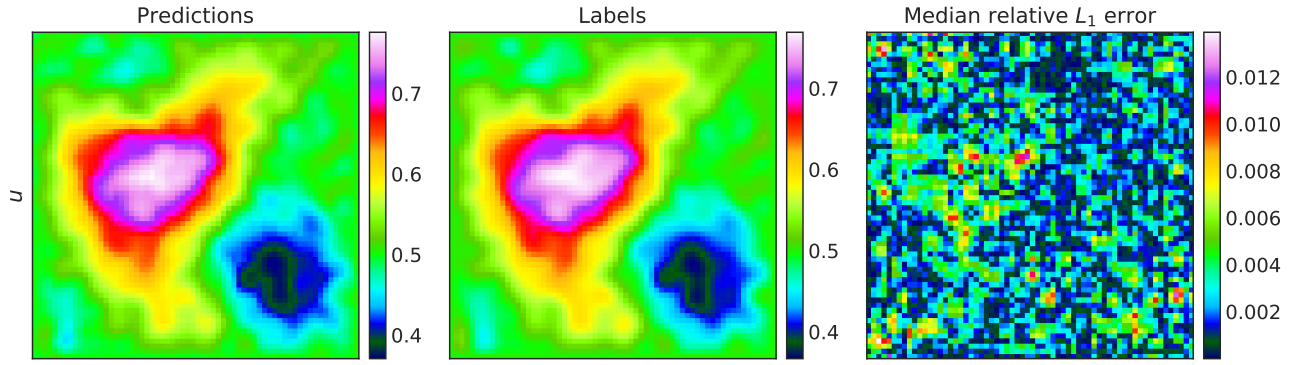


Figure 7: **Wave equation.** Approximation generated by POSEIDON-T using the contrastive learning approach.

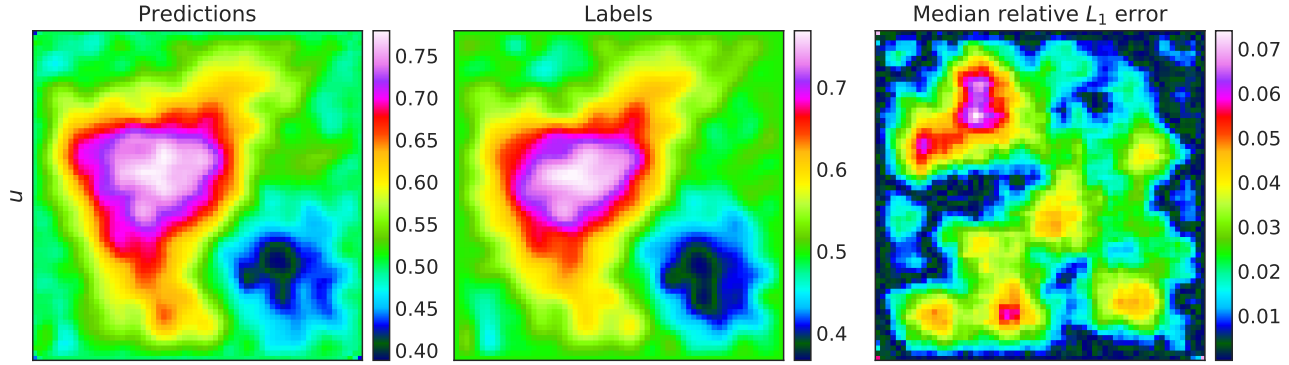


Figure 8: **Wave equation.** Approximation generated by POSEIDON-T using the L_1 regression loss.

Similarly for more complex flow dynamics, the RNC approach also outperforms the L_1 regression loss. Table 2 shows the results of both approaches on the Navier-Stokes equations datasets, see Section 5.1 for details. The RNC approach allows for the more accurate resolution of finer details in the flow structures as seen in Figures 9-10.

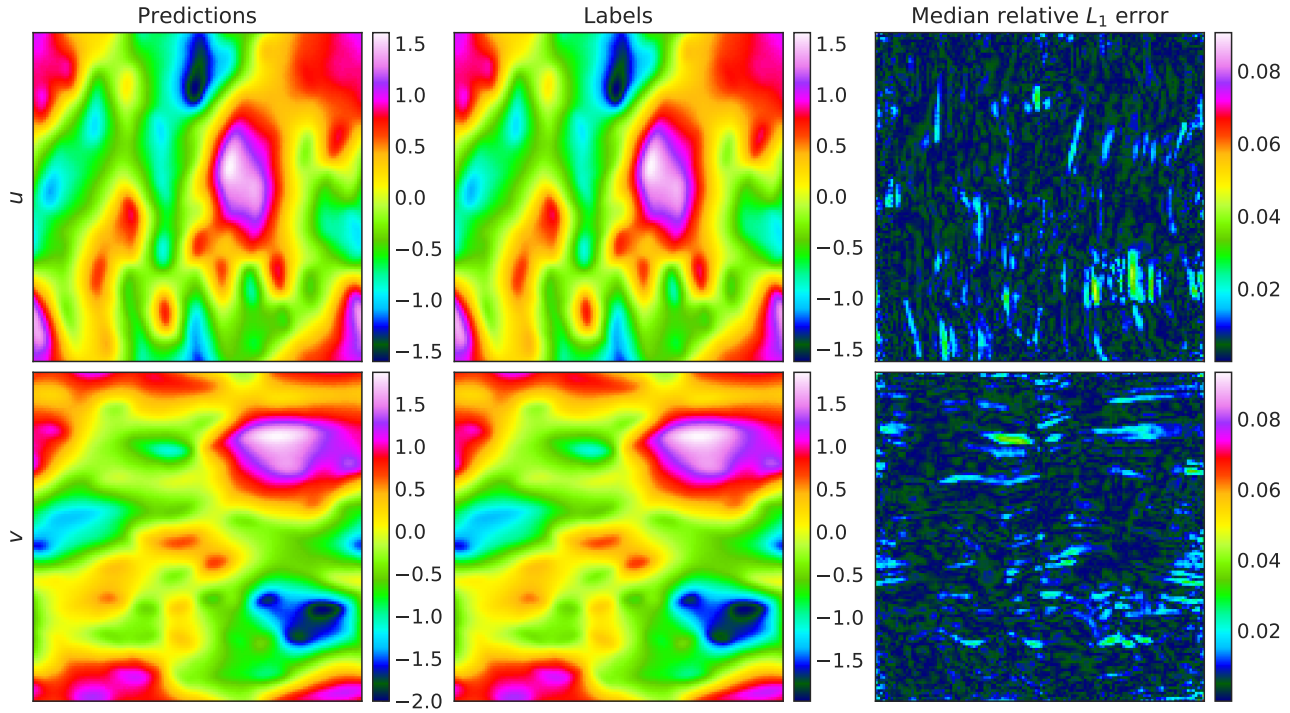


Figure 9: **NS-PwC.** Approximation generated by POSEIDON-T using the contrastive learning approach.

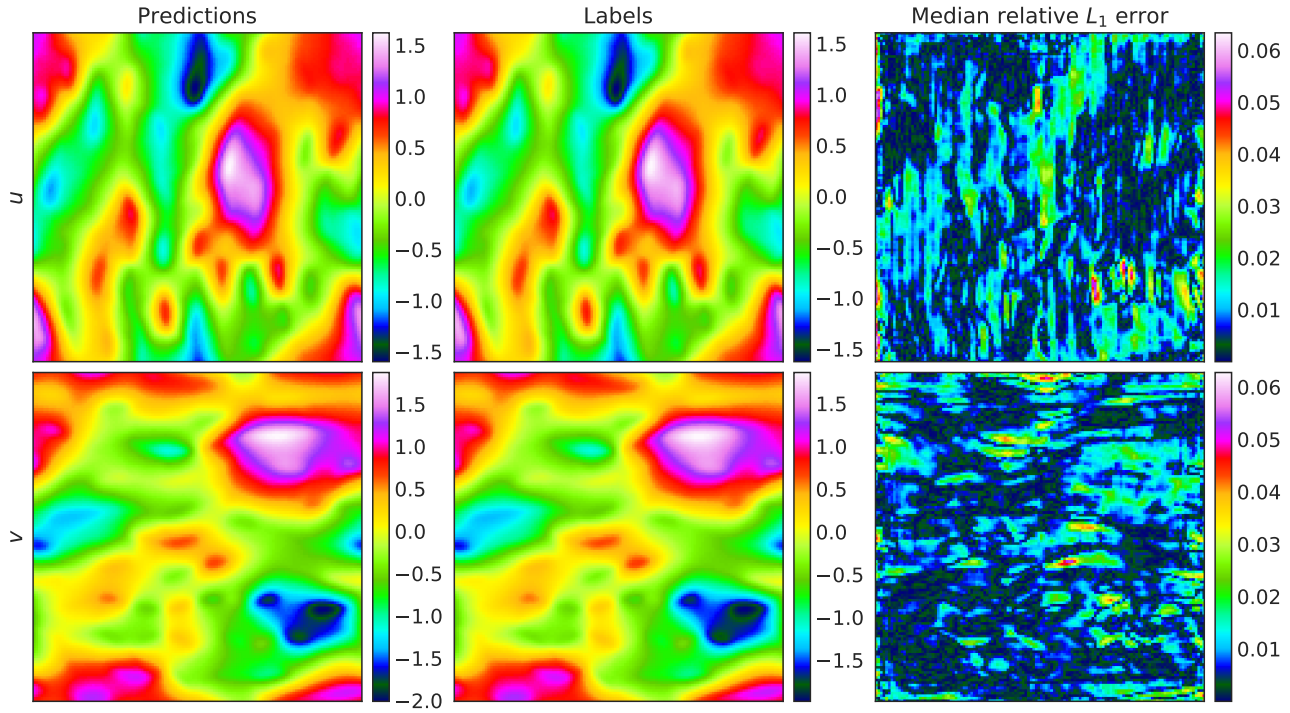


Figure 10: **NS-PwC.** Approximation generated by POSEIDON-T using the L_1 regression loss.

The increased performance of the contrastive learning approach on the OOD predictions suggests that the POSEIDON model is able to learn more *effective and meaningful representations* as compared to using a simple regression loss. This is illustrated by the observed reordering of the latent space as shown in Figure 6 for the Wave equation dataset.

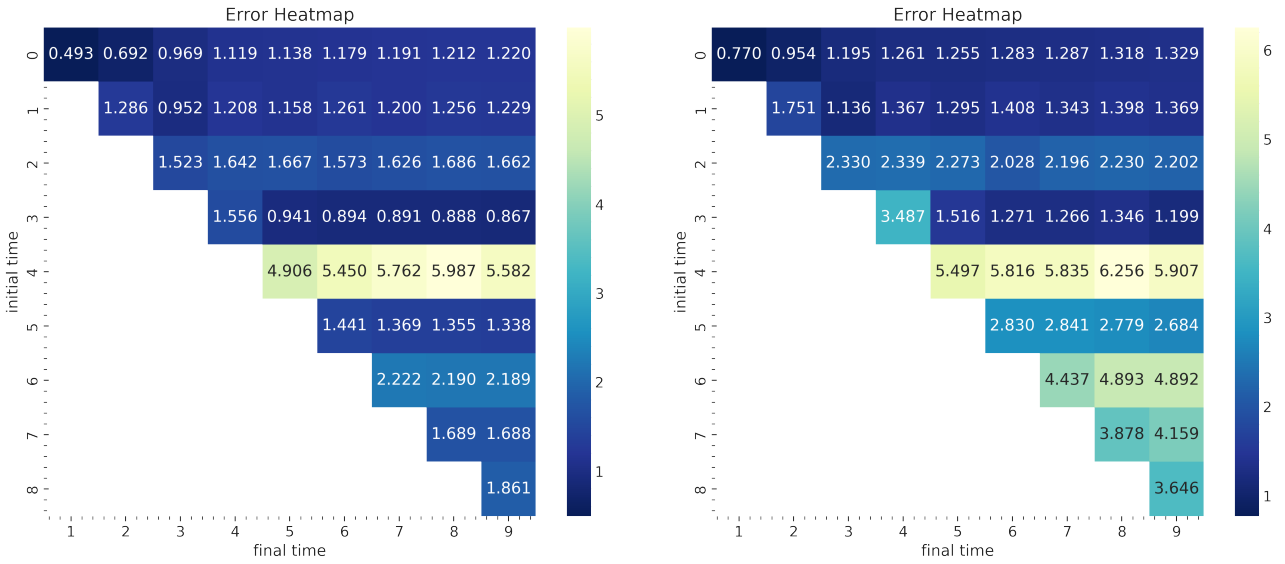
	NS-Gauss (ID)	NS-Sines (ID)	NS-PwC (zero-shot)		
Method	**Average**	**Average**	**Average**	**Min**	**Max**
POSEIDON-T L_1	2.62	3.43	2.34	1.61	3.31
POSEIDON-T $\mathcal{L}_{\text{RNC}}$	2.19	3.25	2.13	1.21	3.28
Gains	**+0.43**	**+0.18**	**+0.21**	**+0.40**	**+0.03**

Table 2: **Incompressible Navier-Stokes equations dataset.** ID and zero-shot comparison between the RNC loss and the L_1 regression loss. The contrastive learning learning approach surpasses the regression loss on all three datasets.

5.4 Analysis and Discussion

Performance by time-step. To further investigate the potential of the contrastive learning approach, I analyzed the temporal prediction accuracy by computing the average L_1 error across the evaluation dataset for different time differences, as shown in Figure 11. The heatmap visualizes prediction error as a function of initial time (y-axis) and prediction target time (x-axis), revealing how model performance varies across different time steps. We clearly see that the contrastive learning approach is beneficial over all time steps.

Reconstruction loss influence. The additional implementation of a reconstruction loss provided significant performance improvement by about 3% of L_1 error on average and was crucial for the whole training approach. This additional L_1 loss ensures that the learned representations retain enough information to reconstruct the original input. Without this constraint, the encoder might learn to discard important features while only focusing on contrastive objectives. The recon loss is especially important for the contrastive learning approach to POSEIDON as the target space is much more complex as the one-dimensional target space used in [6].
Indeed, the contrastive loss and the reconstruction loss try each to *minimize a different objective*, leading to an overall well balanced training. The recon loss especially focuses on preserving fine-grained details and forces to model to retain important features in the data resulting in more robust representations [7]. Figure 12 shows the direct influence of the recon loss on the latent space.



(a) (b)

Figure 11: **Wave equation.** Average L_1 error on ID evaluation dataset. The y-axis represents the initial time step from which predictions are made, while the x-axis shows the target time step being predicted. Color intensity indicates prediction error magnitude, with lighter regions representing higher errors. *Contrastive Loss (a) | Base Model (b)*

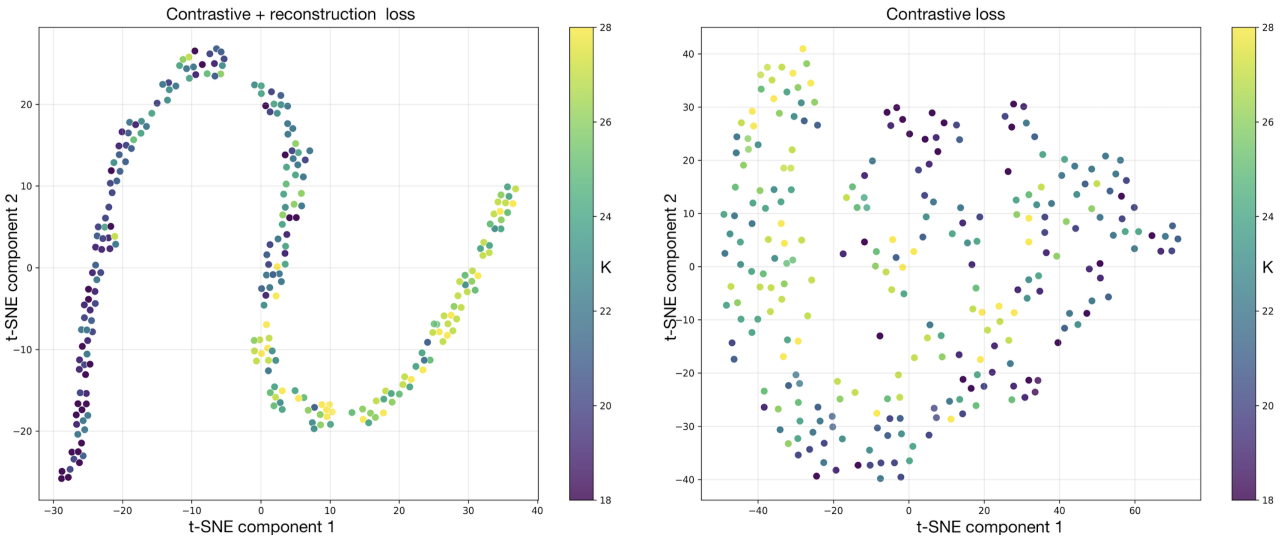


Figure 12: **Learned representations on the Wave equation datasets.** The additional reconstruction loss combined to the contrastive loss helps the model to better capture the continuity in the data.

Influence of the batch size. Large batch sizes are crucial for a successful contrastive learning approach. Each batch has to provide a sufficient number of negative samples for the contrastive objective to work effectively. With small batches the model has fewer negative examples to contrast against, thus making it difficult to learn meaningful representations.

Larger batches create a richer set of negative pairs (see Figure 2) and allow the model to better separate positive pairs from negative pairs, leading to more robust feature learning, better convergence

and better downstream performance. In future work, I wish to further investigate the influence of the batch size on the training performance.

Model size. For my implementation, I considered the smallest variant of the POSEIDON model family. I am convinced that the impact of the contrastive learning approach will be enhanced when switching to large models.
I wish to further investigate in particular the impact of the *patch size* on the learning. Smaller patches would allow the model to better capture fine-grained details enabling the contrasting of low-level feature representations.

Hyperparameter tuning. Hyperparameter tuning is *crucial* for the model performance. My training choices are presented in **SM** A.1. The contrastive learning approach introduced a new parameter, the temperature τ, as well as the reconstruction loss weight. I believe the model could perform better by further tuning these parameters.

Training time. The contrastive learning framework allows for a *significant reduction in training time* as compared to the standard regression loss. As an example, training the POSEIDON-T model on the Navier-Stokes dataset with the specifications detailed in **SM** A.1 took around 15 hours using the contrastive approach against 23 hours using the regression loss. This is due to the model being able to easily process larger batch sizes, which additionally also benefits the contrastive approach as discussed. On the other hand, the regression loss *requires small batch sizes* due to *gradient noise accumulation* becoming a problem for larger batch sizes [13].
Additionally, the new one-stage training approach allowed to reduce the total training time by about one third as compared to the two-stage approach implemented in [6].

6 Conclusion

I implemented a contrastive learning approach within the POSEIDON framework, that allows the model to learn continuous representations for regression. I based my research on the RNC framework presented by Zha et al. [6] that ranks training samples according to their labels before contrasting them against each other based on their relative ranking. Experiments on different datasets of time-dependent fluid flows showed superior performance in terms of accuracy and training time of the contrastive learning approach as compared to a standard regression approach. The contrastive framework showed its ability to learn meaningful embeddings and demonstrated its effectiveness in discovering complex physical relationships and flow structures that were overlooked by the regression method. These findings suggest that contrastive learning could improve the POSEIDON framework as a foundation model for learning the solution operators of PDEs.

Acknowledgments

I would like to thank Siddhartha Mishra for giving me this unique opportunity to join his research group at SAM. I was able to experience international research at the highest level and still finding a very welcoming atmosphere at the lab. I would like to thank Maximilian Herde for his help and advice during this project. It was a true pleasure working together with him.

References

[1] A M Quarteroni and A Valli. "Numerical Approximation of Partial Differential Equations, 1994. 2nd printing." In: *Springer* (2008).

[2] G. E. Karniadakis, I. G. Kevrekidis, and L. Lu. "Physics informed machine learning." In: *Nature Reviews Physics* (2021).

[3] S. Mishra and A. E. Townsend. *Numerical Analysis meets Machine Learning. Handbook of Numerical Analysis*. Springer, 2024.

[4] Rishi Bommasani et al. "On the Opportunities and Risks of Foundation Models". In: (July 2022). URL: http://arxiv.org/abs/2108.07258.

[5] Maximilian Herde et al. "Poseidon: Efficient Foundation Models for PDEs". In: (May 2024). URL: http://arxiv.org/abs/2405.19101.

[6] Kaiwen Zha et al. "Rank-N-Contrast: Learning Continuous Representations for Regression". In: (Oct. 2022). URL: http://arxiv.org/abs/2210.01189.

[7] Dan Hendrycks and Thomas Dietterich. "Benchmarking neural network robustness to common corruptions and perturbations". In: *7th International Conference on Learning Representations, ICLR*. 2019.

[8] Ze Liu et al. "Swin Transformer: Hierarchical Vision Transformer using Shifted Windows". In: *Proceedings of the IEEE International Conference on Computer Vision*. 2021. DOI: 10.1109/ICCV48922.2021.00986.

[9] Hu Cao et al. "Swin-Unet: Unet-Like Pure Transformer for Medical Image Segmentation". In: *Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)*. Vol. 13803 LNCS. 2023. DOI: 10.1007/978-3-031-25066-8{_}9.

[10] Zhuang Liu et al. "A ConvNet for the 2020s". In: *Proceedings of the IEEE Computer Society Conference on Computer Vision and Pattern Recognition*. Vol. 2022-June. 2022. DOI: 10.1109/CVPR52688.2022.01167.

[11] Ting Chen et al. "A simple framework for contrastive learning of visual representations". In: *37th International Conference on Machine Learning, ICML*. Vol. PartF168147-3. 2020.

[12] Kaiming He et al. "Deep residual learning for image recognition". In: *Proceedings of the IEEE Computer Society Conference on Computer Vision and Pattern Recognition*. Vol. 2016-December. 2016. DOI: 10.1109/CVPR.2016.90.

[13] William Merrill et al. “Critical Batch Size Revisited: A Simple Empirical Approach to Large-Batch Language Model Training”. In: (May 2025). URL: http://arxiv.org/abs/2505.23971.

A Appendix

A.1 Parameters

These are the model parameters I used for the computation of the above results:

POSEIDON-T Contrastive Approach

- **Dataset size:** 1024
- **Effective batch size:** 350
- **Number of epochs:** 400
- **Scheduler:** Cosine Decay
- **Maximum learning rate:** $5 \cdot 10^{-3}$
- **Weight Decay:** $1 \cdot 10^{-4}$
- **Early stopping:** Yes
- **Temperature Parameter:** 1.75
- **Recon Loss Weight:** 10%

POSEIDON-T L_1 Regression Loss

- **Dataset size:** 1024
- **Effective batch size:** 32
- **Number of epochs:** 400
- **Scheduler:** Cosine Decay
- **Maximum learning rate:** $1 \cdot 10^{-4}$
- **Weight Decay:** $1 \cdot 10^{-4}$
- **Early stopping:** Yes

A.2 Predictions

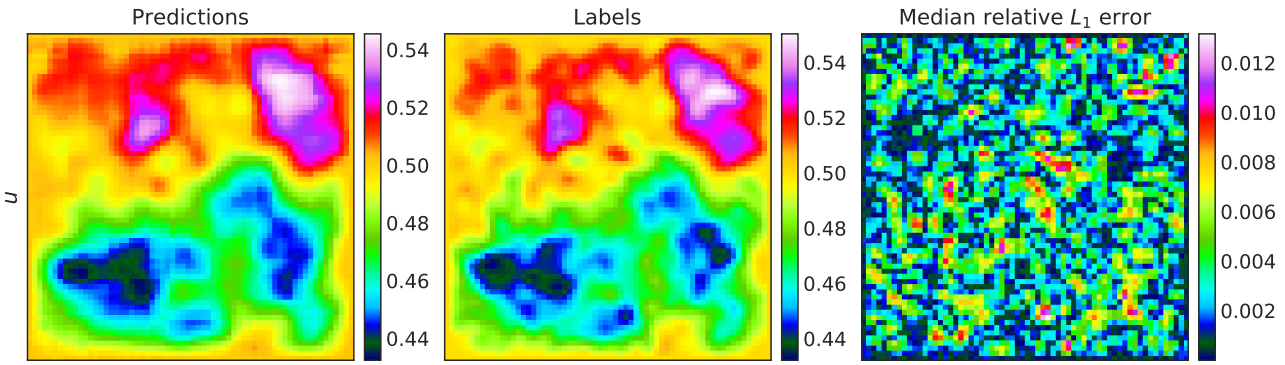


Figure 13: **Wave equation.** Approximation on the OOD dataset generated by POSEIDON-T using the contrastive learning approach.

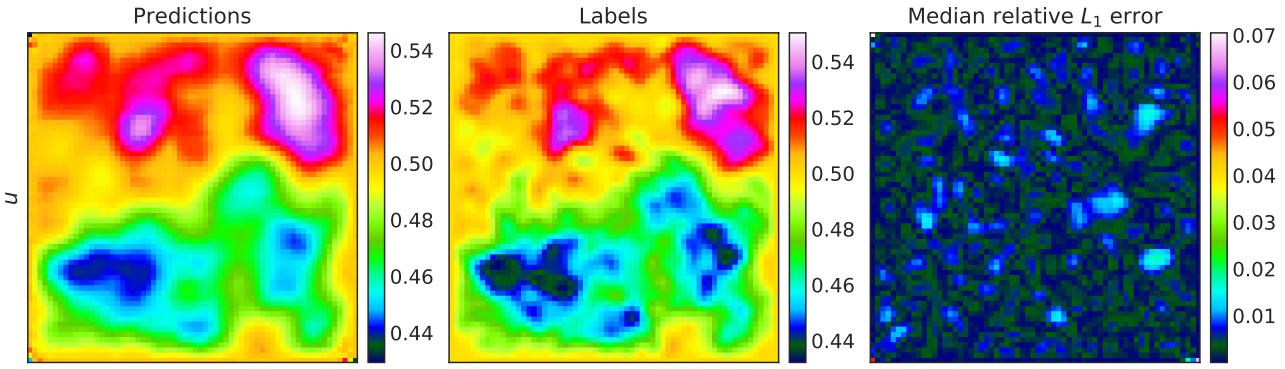


Figure 14: **Wave equation.** Approximation on the OOD dataset generated by POSEIDON-T using the L_1 regression loss.

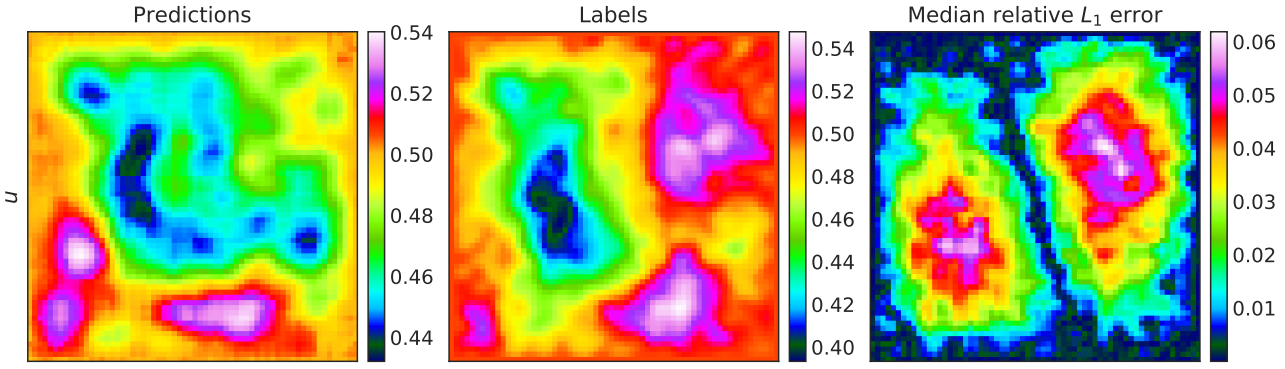


Figure 15: **Wave equation.** Worst approximation on the OOD dataset generated by POSEIDON-T using the contrastive learning approach. The performance is still better than using the L_1 loss.

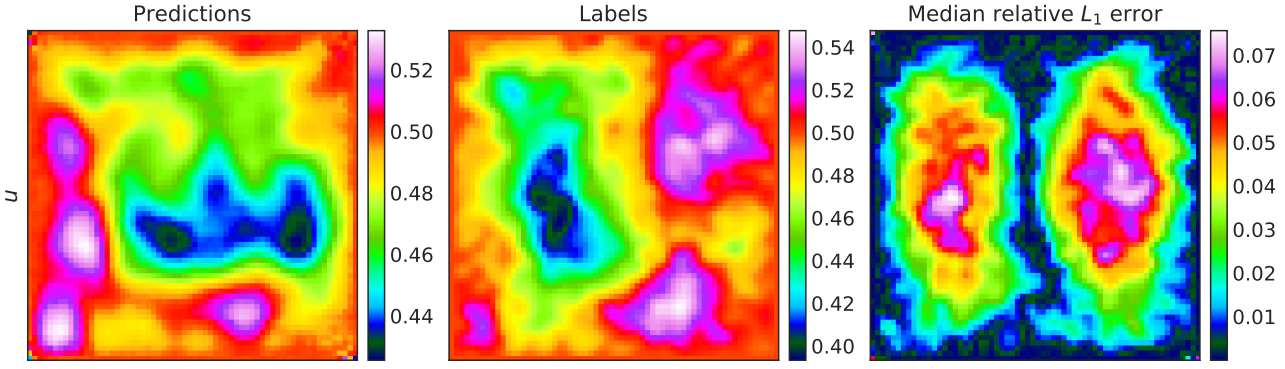


Figure 16: **Wave equation.** Worst approximation on the OOD dataset generated by POSEIDON-T using the L_1 regression loss.